ECE30030/ITP30010 – Database Systems

Relational Data

Reading: Chapter 1-2

Charmgil Hong

charmgil@handong.edu

Spring, 2026

Handong Global University

Course Overview

- Course: ITP30010 Database Systems
  - ITP-section#1 (English): Mon/Thu 4:00-5:15pm @NTH313
  - ITP-section#2 (English): Mon/Thu 5:30-6:45pm @ NTH313

- Instructor: Charmgil Hong (홍참길)
  - Office: NTH201
  - Email: charmgil@handong.edu
  - Office hours: By appointment
    - Schedule is required: https://calendly.com/charmgil/sched

- Teaching Assistants:
  - Yujin Kim (yujin.kim@handong.ac.kr)
  - Yunseong Choi (yunseong.choi@handong.ac.kr)
  - *(One more TA will join)*

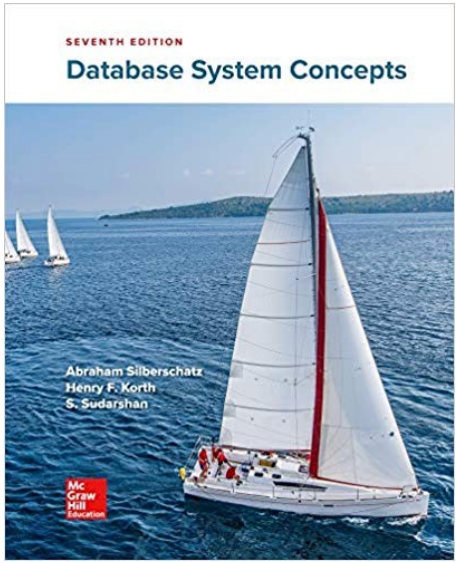
Schedule (tentative)

	Subject		Subject
1	Admin, Introduction DBMS, Relation data model	9	Transactions Midterm
2	Relational algebra Installing a DBMS	10	Transactions Database storages
3	Structured Query Language (DML)	11	Database storages Quiz
4	Structured Query Language (DDL) Quiz	12	Indexes
5	Entity-Rleationship (ER) diagrams	13	Indexes Quiz
6	Normalization theory Quiz	14	Keys, Functions/Procedures, Triggers
7	Advanced SQL	15	Beyond relational data Quiz
8	Advanced SQL, Constraints, Views Quiz	16	Final

Announcements

- Homework assignment #1 will be released
  - Release: This week (Week #2)
  - Due: Two week from the release

  - You will need the textbook
    - Abraham Silberschatz, Henry F. Korth, S. Sudarshan. ***Database System Concepts, 7th edition.*** McGraw Hill. 2019.



Administrivia

- Grading
  - **Midterm: 23%**
  - **Final: 25%**
  - **Quiz: 25%**
  - **Term project: 20%**
  - **Participation: 7%**

Administrivia

- Any of the followings will result in **failure (F)**:
  - **Conducting any form of cheating or academic dishonesty**
  - **Not appearing more than 3/4 of all meetings**
    - **Three times of tardiness will be countered as one absence**
  - **Not taking any of the midterm and final exam**

Administrivia

- Flipped learning
  - In-person lectures are offered once a week
    - Pre-recorded video lectures will be provided on LMS
    - Monday sessions are held for recap and discussion
    - Thursday sessions will be held biweekly for quizzes (starting in Week 4)

- Attendance
  - Attendance does not directly go to your grade
  - If you are absent for more than ¼ classes you will automatically get an “F”

Last Lecture

- Some definitions on DB
- Motivation, Examples
- Brief history

Agenda

- R-DBMS
- Relational Data Model

Database Systems

- Database
  - Organized collection of inter-related data that models some aspect of the real-world (A. Pavlo)
    - Things related are laid together; *c.f.*, files are not like this

- Database system: *Informal definition*

Magnetic tape
(IBM 729 Tape Drive)

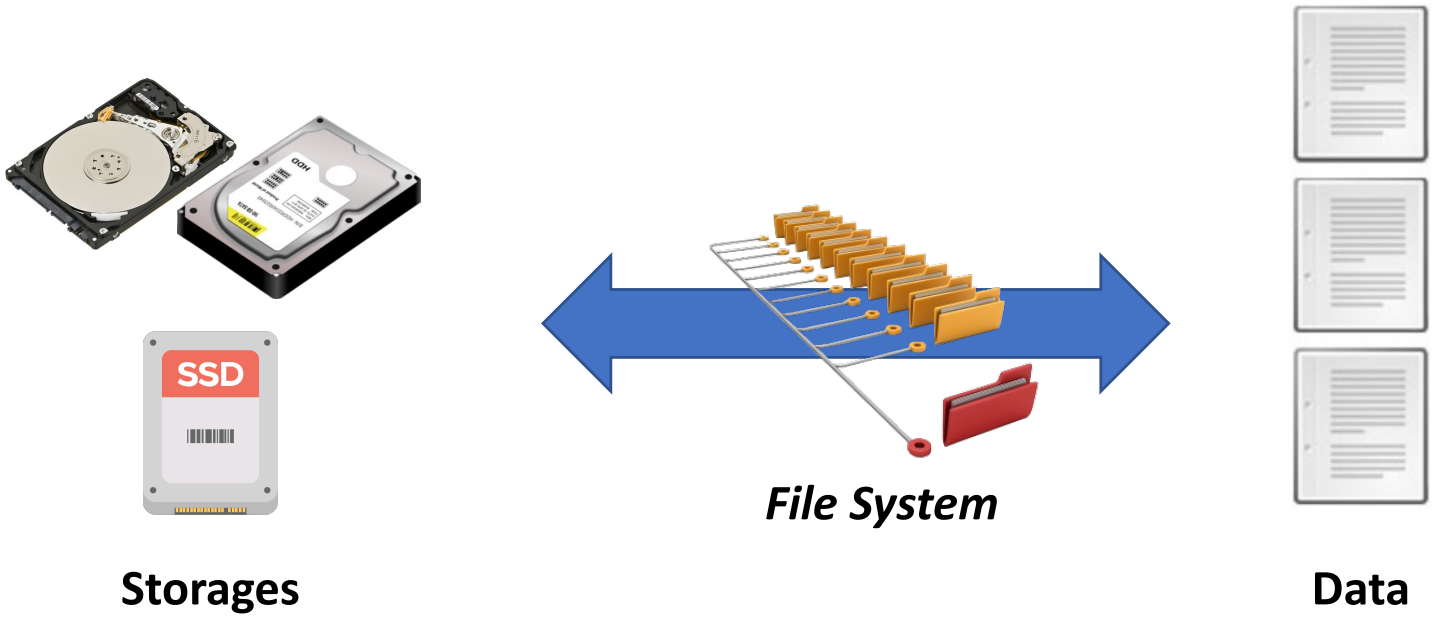
First hard disk drive
(IBM Model 350, 5MB)

* Image src: https://www.youtube.com/shorts/jUyorLotzhw ; https://www.pingdom.com/blog/amazing-facts-and-figures-about-the-evolution-of-hard-disk-drives/

Database Systems

- Database
  - Organized collection of inter-related data that models some aspect of the real-world (A. Pavlo)
    - Things related are laid together; *c.f.*, files are not like this

- Database system: *Informal definition*

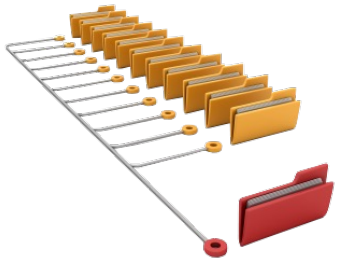


* Image src: https://www.iconfinder.com/icons/4263530/disk_drive_gray_ssd_icon; https://pixabay.com/vectors/hdd-hard-disk-drive-disk-hard-disk-154463/

Database Systems

- Database
  - Organized collection of inter-related data that models some aspect of the real-world (A. Pavlo)
    - Things related are laid together; *c.f.*, files are not like this

- Database system: *Informal definition*



File System

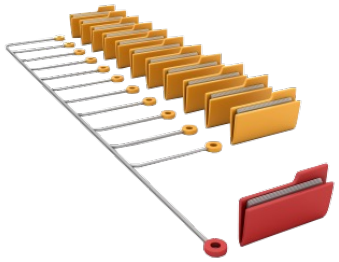
  - Flat file strawman
    - Store a database as comma-separated value (CSV) files
    - Manage the CSV files using our own code
      - Use a separate file per entity
      - The applications have to parse the CSV files each time they want to read or update records

* Image src: https://www.iconfinder.com/icons/4263530/disk_drive_gray_ssd_icon; https://pixabay.com/vectors/hdd-hard-disk-drive-disk-hard-disk-154463/

Database Systems

- Database
  - Organized collection of inter-related data that models some aspect of the real-world (A. Pavlo)
    - Things related are laid together; *c.f.*, files are not like this

- Database system: *Informal definition*



File System

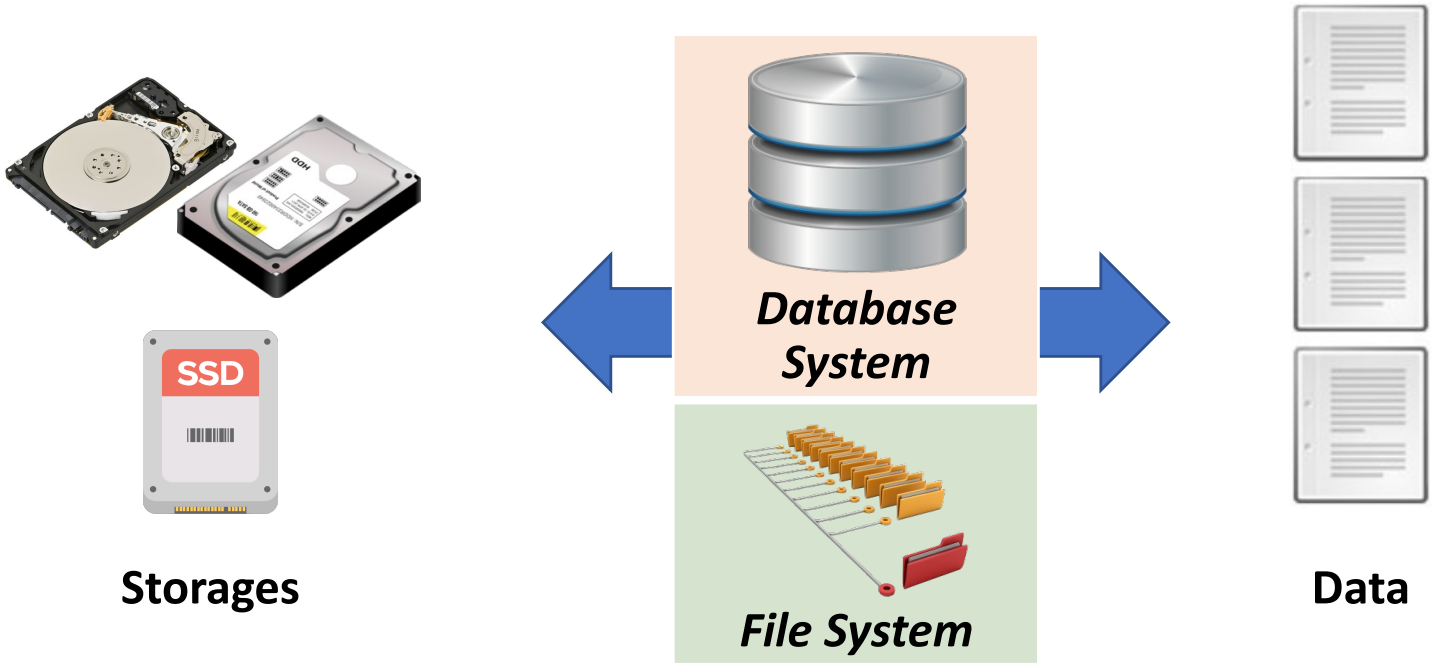
  - Flat file strawman
    - Issue: data integrity
      - How to examine the validity of the values?
    - Issue: implementation
      - How to find a particular record?
      - How to write a new application that uses the same data
    - Issue: durability
      - What if the machine crashes while file writing?

* Image src: https://www.iconfinder.com/icons/4263530/disk_drive_gray_ssd_icon; https://pixabay.com/vectors/hdd-hard-disk-drive-disk-hard-disk-154463/

Database Systems

- Database
  - Organized collection of inter-related data that models some aspect of the real-world (A. Pavlo)
    - Things related are laid together; *c.f.*, files are not like this

- Database system: *Informal definition*



* Image src: https://www.iconfinder.com/icons/4263530/disk_drive_gray_ssd_icon; https://pixabay.com/vectors/hdd-hard-disk-drive-disk-hard-disk-154463/

Database Systems

- Database management system (DBMS)
  - Software that allows applications to store and analyze information in a database
    - Access data without worrying about the file I/O-level details
  - A general-purpose DBMS is designed to allow the definition, creation, querying, update, and administration of databases

Database Systems

- DBMS as a data storage
  - Database abstraction to avoid low-level implementation and maintenance chores
    - Store database in simple data structures
    - Access data through high-level language
  - Database abstraction does not include:
    - How to implement the storage, relations, …
    - Clear separation between logical vs. physical layers

- DBMS as an interface
  - Data definition language (DDL)
  - Data manipulation language (DML)

  → Structured query language (SQL) includes both DDL and DML

Agenda

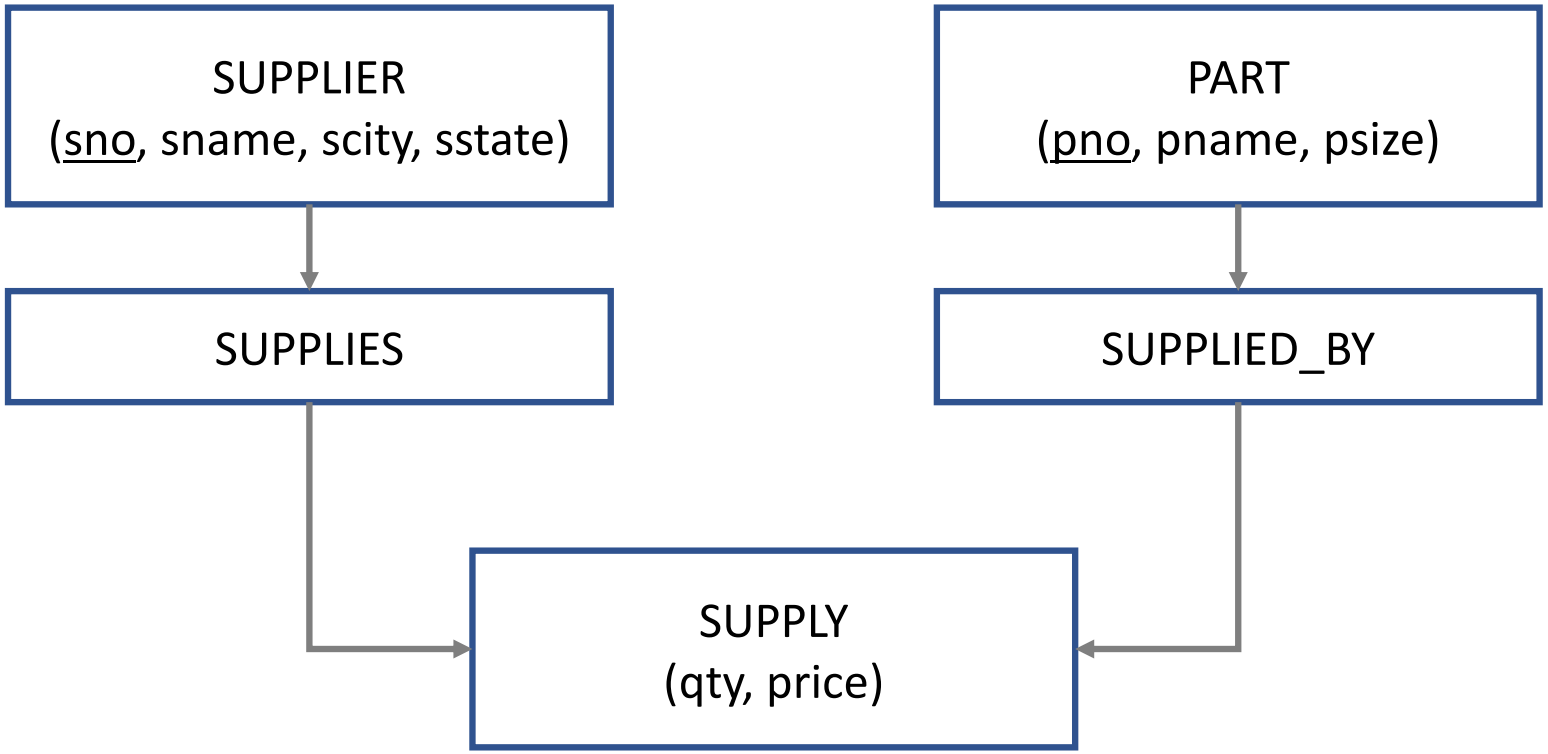
- R-DBMS
- **Relational Data Model**

Data Model

- Data model: A notion for describing data or information
  - Data model consists of three parts:
    - Structure
    - Operations
    - Constraints
  - Examples
    - Relational data model: the most conventional ← *main focus of the course*!

Network Data Model

- Example: Schema



Example taken from: A. Pavlo. 15-721 Advanced Database Systems. https://15721.courses.cs.cmu.edu/spring2020/slides/01-history.pdf

Network Data Model

- Example: Instances

SUPPLIER

sno	sname	scity	sstate
1001	Liquid Dynamo	New York	NY
1002	Motions	Boston	MA

PART

pno	pname	psize
501	Gas Cylinders	Large

SUPPLIES

parent	child

SUPPLIED_BY

parent	child

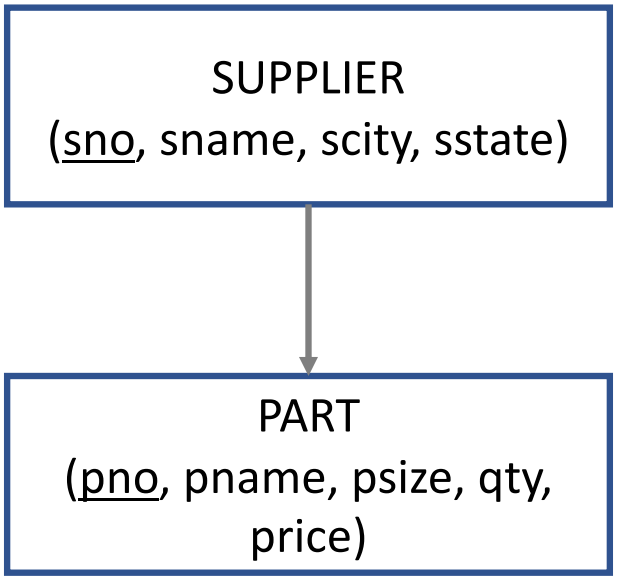
SUPPLY

qty	price
20	$185
15	$200

Example taken from: A. Pavlo. 15-721 Advanced Database Systems. https://15721.courses.cs.cmu.edu/spring2020/slides/01-history.pdf

Hierarchical Data Model

- Example: Schema & Instances



sno	sname	scity	sstate	parts
1001	Liquid Dynamo	New York	NY	
1002	Motions	Boston	MA	

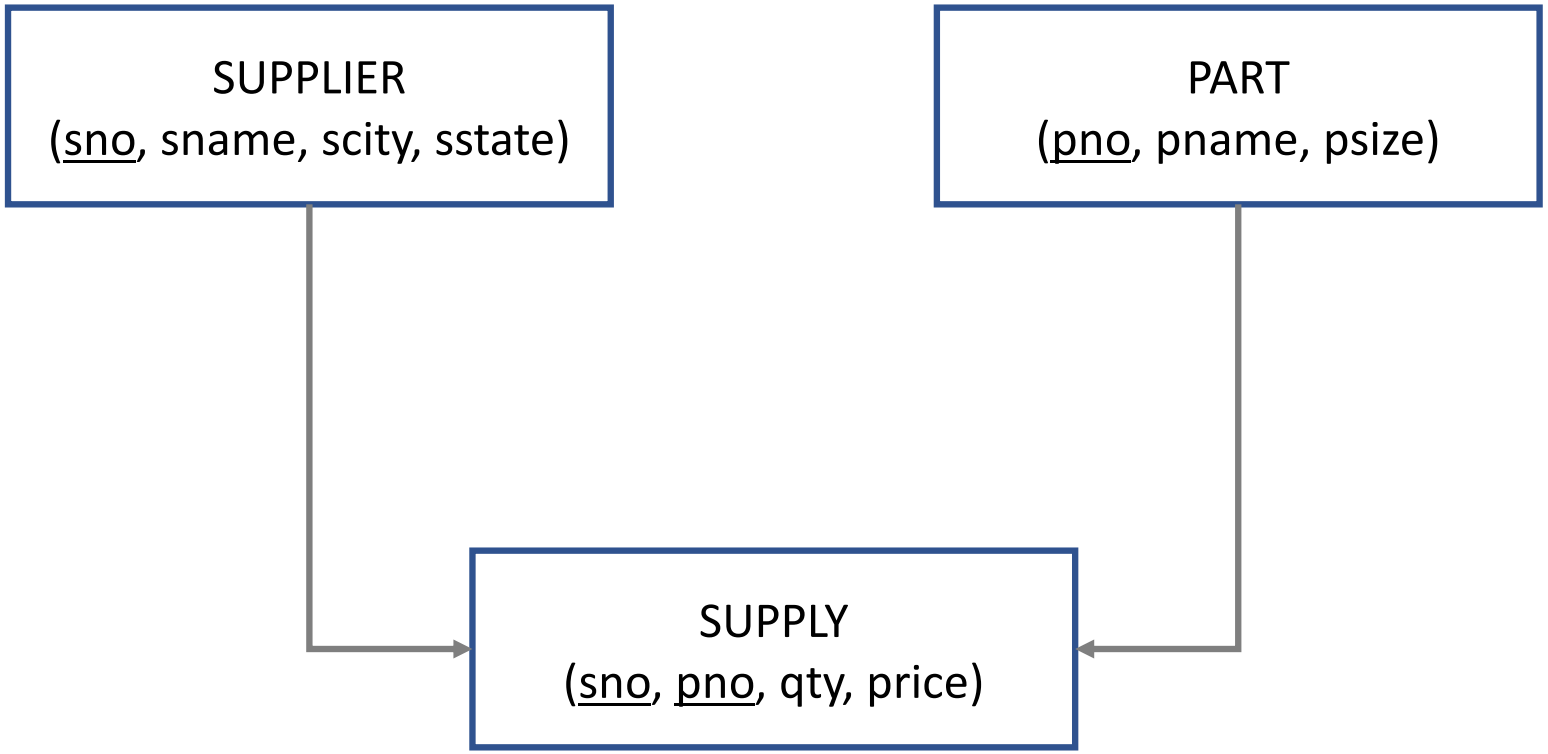
pno	pname	psize	qty	price
501	Gas Cylinders	Large	20	$185

pno	pname	psize	qty	price
501	Gas Cylinders	Large	15	$200

Example taken from: A. Pavlo. 15-721 Advanced Database Systems. https://15721.courses.cs.cmu.edu/spring2020/slides/01-history.pdf

Relational Data Model

- Example: Schema



Example taken from: A. Pavlo. 15-721 Advanced Database Systems. https://15721.courses.cs.cmu.edu/spring2020/slides/01-history.pdf

Relational Data Model

- Example: Instances

SUPPLIER

sno	sname	scity	sstate
1001	Liquid Dynamo	New York	NY
1002	Motions	Boston	MA

PART

pno	pname	psize
501	Gas Cylinders	Large

SUPPLY

sno	pno	qty	price
1001	501	20	$185
1002	501	15	$200

Example taken from: A. Pavlo. 15-721 Advanced Database Systems. https://15721.courses.cs.cmu.edu/spring2020/slides/01-history.pdf

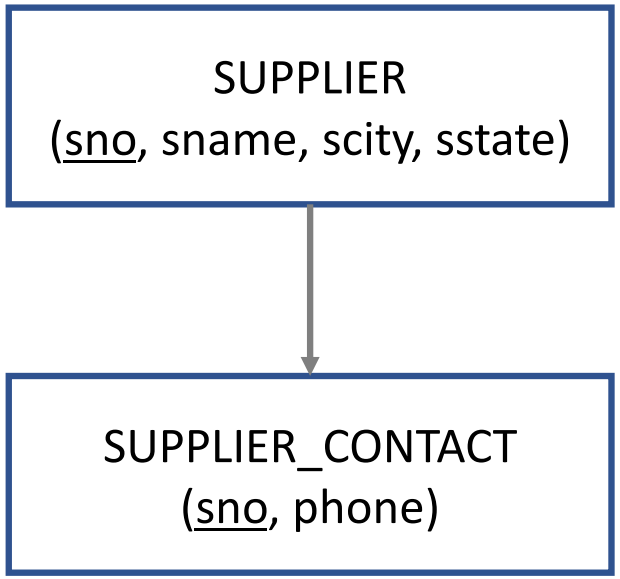
Object-Oriented Data Model

- Example: Application code & Schema

```
Class Supplier {
    int sno;
    String sname;
    String scity;
    String sstate;
    String phone[];
}
```

sno	sname	scity	sstate
1001	Liquid Dynamo	New York	NY

sno	phone
1001	212-111-2222
1001	917-333-4444



Example taken from: A. Pavlo. 15-721 Advanced Database Systems. https://15721.courses.cs.cmu.edu/spring2020/slides/01-history.pdf

Object-Oriented Data Model

- Example: Application code & Object

```
Class Supplier {
    int sno;
    String sname;
    String scity;
    String sstate;
    String phone[];
}
```

Supplier

```
{
    “sno”: 1001,
    “sname”: “Liquid Dynamo”,
    “scity”: “New York”,
    “sstate”: “NY”,
    “phone”: [
        “212-111-2222”,
        “917-333-4444”
    ]
}
```

Example taken from: A. Pavlo. 15-721 Advanced Database Systems. https://15721.courses.cs.cmu.edu/spring2020/slides/01-history.pdf

Data Model

- Data model: A notion for describing data or information
  - Data model consists of three parts:
    - Structure
    - Operations
    - Constraints
  - Examples
    - Relational data model: the most conventional ← *main focus of the course*!
    - NoSQL
      - Key/value
      - Graph
      - Document
      - Column-family
    - Machine learning
      - Array/matrix
    - Misc.: hierarchical, network

Relational Data Model

- Relational data model: A data model describes data in terms of relations

- Relation
  - An unordered set that contains the relationship of attributes that represent entities

Relation (Table)

- Attribute (column)
  - Attribute values are required to be atomic (indivisible data type)
    - String is an atomic data type in most database systems
  - The set of allowed values for each attribute is called the domain of the attribute
  - NULL is a member of every domain, indicating that the value is "unknown"
    - The NULL values cause complications in many operations

- Tuple (row)
  - A tuple is a set of attribute values (also known as its domain) in the relation
  - Each tuple has one value for each attribute of the relation
  - Values are (normally) atomic/scalar

Example: a *Relation*

- *n*-ary relation = table with *n* columns

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Example: a *Relation*

- *n*-ary relation = table with *n* columns

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

4 attributes (columns)

Example: a *Relation*

- *n*-ary relation = table with *n* columns

Header

12 tuples (rows, or records)

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Relation (Table)

- Attribute (column)
  - Attribute values are required to be atomic (indivisible data type)
    - String is an atomic data type in most database systems
  - The set of allowed values for each attribute is called the domain of the attribute
  - NULL is a member of every domain, indicating that the value is "unknown"
    - The NULL values cause complications in many operations

- Tuple (row)
  - A tuple is a set of attribute values (also known as its domain) in the relation
  - Each tuple has one value for each attribute of the relation
  - Values are (normally) atomic/scalar

Relation (Table)

- Relations are unordered: Order of tuples is irrelevant (tuples may be stored in an arbitrary order)
  - Example

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

ID	name ▲ 2	dept_name ▲ 1	salary
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
45565	Katz	Comp. Sci.	75000.00
10101	Srinivasan	Comp. Sci.	65000.00
98345	Kim	Elec. Eng.	80000.00
76543	Singh	Finance	80000.00
12121	Wu	Finance	90000.00
58583	Califieri	History	62000.00
32343	El Said	History	60000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
33456	Gold	Physics	87000.00

Notations

- Using a table

ID	name 2	dept_name 1	salary
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
45565	Katz	Comp. Sci.	75000.00

- Using a set notation

  Structure: *instructor*(*ID, name, dept_name, salary*),

  Tuples: (*76766, Crick, Biology, 72000.00*),
  (*83821, Brandt, Comp. Sci., 92000.00*),
  (*45565, Katz, Comp. Sci., 75000.00*)

  - Mathematically, sets do not have orders nor duplicates
  - However, we implicitly treat each tuple as an ordered set
    - (*76766, Crick, Biology, 72000.00*) != (*72000.00, Biology, Crick, 76766*)

Keys

- Key
  - One type of constraints
  - One or more attributes form a key
  - A key for a relation → do NOT allow duplicates of the same values of the key attributes

Keys

- Definitions
  - Let $K \subseteq R$
  - K is a superkey of R if values for K are sufficient to identify a unique tuple of each possible relation $r(R)$
    - Example:
  - Superkey K is a candidate key if K is minimal
    - Example:
  - One of the candidate keys is selected to be the primary key
    - Which one to choose?

Primary Keys

- A relation's primary key uniquely identifies a single tuple
- Some DBMSs automatically create an internal primary key if you do not define one
  - *E.g.*, SQL:2003 (SEQUENCE), MySQL (AUTO_INCREMENT)

- Example
  - instructor(<u>ID</u>, name, dept_name, salary)

ID	name ▲ 2	dept_name ▲ 1	salary
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
45565	Katz	Comp. Sci.	75000.00

Foreign Keys

- A foreign key specifies that an attribute from one relation has to map to a tuple in another relation
  - Value in one relation must appear in another relation
    - Referencing relation → Referenced relation

- Example

Relation: instructor

ID	name	dept_name	salary
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
45565	Katz	Comp. Sci.	75000.00

Relation: department

dept_name	building	budget
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00

Data Language

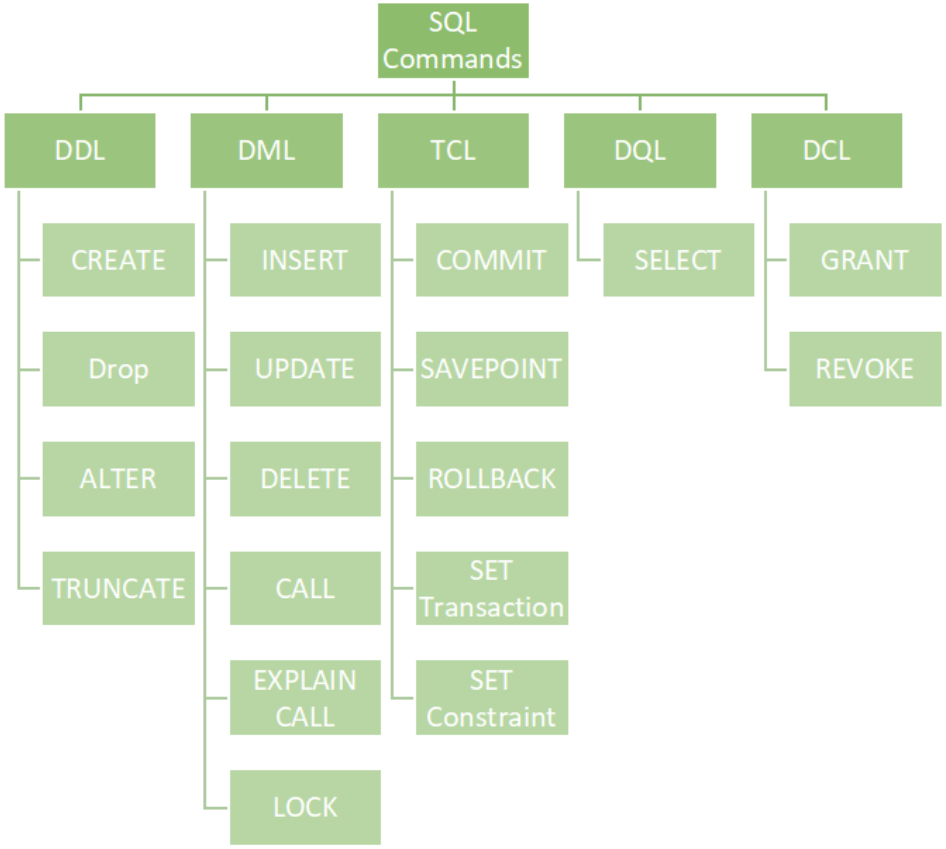
- Data definition language (DDL)
  - How to represent relations and information in a database
    - Defines database schemas

- Data manipulation language (DML)
  - How to store and retrieve information from a database
  - Procedural
    - The query specifies the (high-level) strategy the DBMS should use to find the desired results
    - Based on relational algebra

  - *C.f.*, there are non-procedural DML
    - The query specifies only what data is wanted and not how to find it
    - Based on relational calculus – *this is related to query optimization*

Data Language

- A bit more specific …
  - DDL
  - DML
  - TCL: Transaction Control Lang.
  - DQL: Data Query Lang.
  - DCL: Data Control Lang.



Database Schema

- Database: a collection of relations (tables)
- Database schema: the logical structure of the database
- Database instance: a snapshot of the data in the database at a given instant in time
  - Relation instance: a snapshot of a relation (attributes and tuples) at a given instant in time

ECE30030/ITP30010 Database Systems

Relational Algebra

Reading: Chapter 2

Charmgil Hong

charmgil@handong.edu

Spring, 2026

Handong Global University

Agenda

- Relational algebra
  - Select
  - Project
  - Cartesian product
  - Join
  - Rename
  - Union
  - Set-intersection
  - Set-difference

Algebra

- Mathematical system consisting of
  - Operands: variables or values from which new values can be constructed
  - Operators: symbols denoting procedures that construct new values from given operands

Relational Algebra

- A procedural language consisting of a set of operations that take one or two relations as input and produce a new relation as their output

- Basic operators
  - Select: σ
  - Project: $\prod$
  - Cartesian product: $\times$
  - Join: $\bowtie$
  - Rename: ρ

  - Union: $\cup$
  - Set-intersection: $\cap$
  - Set-difference: $-$

Two Example Relations

- Throughout this module, we will use the following two example relations to illustrate the concepts

Instructor relation

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

teaches relation

ID	course_id	sec_id	semester	year
76766	BIO-101	1	Summer	2017
76766	BIO-301	1	Summer	2018
10101	CS-101	1	Fall	2017
45565	CS-101	1	Spring	2018
83821	CS-190	1	Spring	2017
83821	CS-190	2	Spring	2017
10101	CS-315	1	Spring	2018
45565	CS-319	1	Spring	2018
83821	CS-319	2	Spring	2018
10101	CS-347	1	Fall	2017
98345	EE-181	1	Spring	2017
12121	FIN-201	1	Spring	2018
32343	HIS-351	1	Spring	2018
15151	MU-199	1	Spring	2018
22222	PHY-101	1	Fall	2017

Select Operation

- The select operation selects tuples that satisfy a given predicate
- Notation: $\sigma_p(r)$
  - p is called the selection predicate

Select Operation

- The select operation selects tuples that satisfy a given predicate
- Notation: $\sigma_p(r)$
  - p is called the selection predicate

- Example: select those tuples of the instructor relation where the instructor is in the "Comp. Sci." department
  - Query: $\sigma_{\text{dept_name="Comp. Sci."}}(\text{instructor})$

Select Operation

- The select operation selects tuples that satisfy a given predicate
- Notation: $\sigma_p(r)$
  - p is called the selection predicate

- Example: select those tuples of the instructor relation where the instructor is in the "Comp. Sci." department
  - Query: $\sigma_{\text{dept_name="Comp. Sci."}}(\text{instructor})$

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Select Operation

- The select operation selects tuples that satisfy a given predicate
- Notation: $\sigma_p(r)$
  - p is called the selection predicate

- Example: select those tuples of the instructor relation where the instructor is in the "Comp. Sci." department
  - Query: $\sigma_{\text{dept_name="Comp. Sci."}}(\text{instructor})$

  - Result

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000.00
45565	Katz	Comp. Sci.	75000.00
83821	Brandt	Comp. Sci.	92000.00

Select Operation

- Comparisons using $=, \neq, >, \geq, <, \leq$ are allowed in the selection predicates
- Combine several predicates into a larger predicate using the connectives: $\wedge$ (**and**), $\vee$ (**or**), $\neg$ (**not**)

Select Operation

- Comparisons using =, ≠, >, ≥, <, ≤ are allowed in the selection predicates
- Combine several predicates into a larger predicate using the connectives: ∧ (**and**), ∨ (**or**), ¬ (**not**)

- Example: Find the instructors in Comp. Sci. with a salary greater than $70,000
  - Query: $\sigma_{\text{dept_name="Comp. Sci."} \wedge \text{salary>70,000}}(\text{instructor})$

Select Operation

- Comparisons using =, ≠, >, ≥, <, ≤ are allowed in the selection predicates
- Combine several predicates into a larger predicate using the connectives: ∧ (**and**), ∨ (**or**), ¬ (**not**)

- Example: Find the instructors in Comp. Sci. with a salary greater than $70,000
  - Query: $\sigma_{\text{dept_name="Comp. Sci."} \wedge \text{salary>70,000}}(\text{instructor})$

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Select Operation

- Comparisons using =, ≠, >, ≥, <, ≤ are allowed in the selection predicates
- Combine several predicates into a larger predicate using the connectives: ∧ (**and**), ∨ (**or**), ¬ (**not**)

- Example: Find the instructors in Comp. Sci. with a salary greater than $70,000
  - Query: $\sigma_{\text{dept_name="Comp. Sci."} \wedge \text{salary>70,000}}(\text{instructor})$

  - Result

ID	name	dept_name	salary
45565	Katz	Comp. Sci.	75000.00
83821	Brandt	Comp. Sci.	92000.00

Select Operation

- Example: Find all departments whose name is the same as their building name
  - Query: $\sigma_{dept_name=building}(department)$

Project Operation

- A *unary* operation that returns its argument relation, with certain attributes left out
  - Notation: $\prod_{A_1,A_2,A_3,\ldots,A_k}(r)$
    - $A_1,A_2,A_3,\ldots,A_k$ are attribute names and r is a relation name
  - The result is defined as a relation with k columns
    - Columns that are not listed among $A_1,A_2,A_3,\ldots,A_k$ are also removed in the result
    - Duplicate rows are removed from the result (because the resulting relations are sets)

Project Operation

- Example: eliminate the ID and dept_name attributes of instructor
  - Query: $\Pi_{name,\ salary}(instructor)$
  - Result:

Original relation

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Project Operation

- Example: eliminate the dept_name attribute of instructor
  - Query: $\Pi_{name,\ salary}(instructor)$
  - Result:

Projected relation

name	salary
Srinivasan	65000.00
Wu	90000.00
Mozart	40000.00
Einstein	95000.00
El Said	60000.00
Gold	87000.00
Katz	75000.00
Califieri	62000.00
Singh	80000.00
Crick	72000.00
Brandt	92000.00
Kim	80000.00

Original relation

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Composition of Relational Operations

- Relational-algebra operations can be composed together into a relational-algebra expression
  - Recall that the result of a relational-algebra is a relation
  - Instead of giving the name of a relation as the argument of the projection operation, one can give an expression that evaluates to a relation

- Consider the following query: Find the names of all instructors in the Comp. Sci. department
  - Query: $\Pi_{name}(\sigma_{dept_name=\text{“Comp. Sci”}}(instructor))$

Cartesian-Product Operation

- The Cartesian-product operation (denoted by ×) combines information from any two relations
  - Construct a relation of the result out of each possible pair of tuples
- Example: the Cartesian product of the relations *instructor* and *teaches*
  - Query: *instructor* × *teaches*

Instructor relation

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

teaches relation

ID	course_id	sec_id	semester	year
76766	BIO-101	1	Summer	2017
76766	BIO-301	1	Summer	2018
10101	CS-101	1	Fall	2017
45565	CS-101	1	Spring	2018
83821	CS-190	1	Spring	2017
83821	CS-190	2	Spring	2017
10101	CS-315	1	Spring	2018
45565	CS-319	1	Spring	2018
83821	CS-319	2	Spring	2018
10101	CS-347	1	Fall	2017
98345	EE-181	1	Spring	2017
12121	FIN-201	1	Spring	2018
32343	HIS-351	1	Spring	2018
15151	MU-199	1	Spring	2018
22222	PHY-101	1	Fall	2017

Relation: *instructor* × *teaches*

- Example: the Cartesian product of the relations *instructor* and t*eaches*
  - Result (total 180 tuples = 12 instructors x 15 courses)

instructor.ID	name	dept_name	salary	teaches.ID	course_id	sec_id	semester	year
10101	Srinivasan	Comp. Sci.	65000	76766	BIO-101	1	Summer	2017
12121	Wu	Finance	90000	76766	BIO-101	1	Summer	2017
15151	Mozart	Music	40000	76766	BIO-101	1	Summer	2017
22222	Einstein	Physics	95000	76766	BIO-101	1	Summer	2017
32343	El Said	History	60000	76766	BIO-101	1	Summer	2017
…	…	…	…	…	…	…	…	…
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2017
15151	Mozart	Music	40000	10101	CS-101	1	Fall	2017
22222	Einstein	Physics	95000	10101	CS-101	1	Fall	2017
32343	El Said	History	60000	10101	CS-101	1	Fall	2017
…	…	…	…	…	…	…	…	…
…	…	…	…	…	…	…	…	…
10101	Srinivasan	Comp. Sci.	65000	83821	CS-190	2	Spring	2017
12121	Wu	Finance	90000	83821	CS-190	2	Spring	2017
15151	Mozart	Music	40000	83821	CS-190	2	Spring	2017
…	…	…	…	…	…	…	…	…
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2018
15151	Mozart	Music	40000	10101	CS-315	1	Spring	2018
…	…	…	…	…	…	…	…	…
…	…	…	…	…	…	…	…	…

Join Operation

- The Cartesian-Product associates every tuple of *instructor* with every tuple of *teaches*
  - In the previous example, most of the resulting rows have information about instructors who did NOT teach a particular course
- Example: Get only those tuples of "instructor × teaches" that pertain to the courses that the instructor taught
  - Query: $\sigma_{instructor.id=teaches.id}(instructor \times teaches)$

Join Operation

- Example: Get only those tuples of “instructor × teaches” that pertain to the courses that the instructor taught
  - Result

instructor.ID	name	dept_name	salary	teaches.ID	course_id	sec_id	semester	year
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
32343	El Said	History	60000	32343	HIS-351	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-101	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-319	1	Spring	2018
76766	Crick	Biology	72000	76766	BIO-101	1	Summer	2017
76766	Crick	Biology	72000	76766	BIO-301	1	Summer	2018
83821	Brandt	Comp. Sci.	92000	83821	CS-190	1	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-190	2	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-319	2	Spring	2018
98345	Kim	Elec. Eng.	80000	98345	EE-181	1	Spring	2017

Join Operation

- The join operation combines a select operation and a Cartesian-Product operation into a single operation
- Consider relations $r(R)$ and $s(S)$
  - Let θ be a predicate on attributes in the schema R "union" S
  - The join operation $r \bowtie_\theta s$ is defined as follows:

    $$r \bowtie_\theta s = \sigma_\theta (r \times s)$$

- Example: $\sigma_{instructor.id=teaches.id}(instructor \times teaches)$ is equivalent to

$$instructor \bowtie_{Instructor.id=teaches.id} teaches$$

Join Operation

- Example: Get only those tuples of “instructor × teaches” that pertain to the courses that the instructor taught
  - Result

instructor.ID	name	dept_name	salary	teaches.ID	course_id	sec_id	semester	year
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2017
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2018
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2017
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2018
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2018
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2017
32343	El Said	History	60000	32343	HIS-351	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-101	1	Spring	2018
45565	Katz	Comp. Sci.	75000	45565	CS-319	1	Spring	2018
76766	Crick	Biology	72000	76766	BIO-101	1	Summer	2017
76766	Crick	Biology	72000	76766	BIO-301	1	Summer	2018
83821	Brandt	Comp. Sci.	92000	83821	CS-190	1	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-190	2	Spring	2017
83821	Brandt	Comp. Sci.	92000	83821	CS-319	2	Spring	2018
98345	Kim	Elec. Eng.	80000	98345	EE-181	1	Spring	2017

Union Operation

- The union operation combines two relations as a superset of both
  - Notation: $r \cup s$

- For $r \cup s$ to be valid,
  1. *r*, *s* must have the *same* number of attributes (same arity)
  2. The attribute domains must be compatible
     - *E.g.*, the 2nd column of *r* deals with the same type of values as does the 2nd column of *s*

Union Operation

- The union operation combines two relations as a superset of both
  - Notation: $r \cup s$

- For $r \cup s$ to be valid,
  1. *r*, *s* must have the *same* number of attributes (same arity)
  2. The attribute domains must be compatible
     - *E.g.*, the 2nd column of *r* deals with the same type of values as does the 2nd column of *s*

- Example: Find all courses taught in the Fall 2017 semester, or in the Spring 2018 semester, or in both
  - Query: $\Pi_{course_id}(\sigma_{semester=\text{“Fall”} \wedge year=2017}(teaches)) \cup$
    $\Pi_{course_id}(\sigma_{semester=\text{“Spring”} \wedge year=2018}(teaches))$

Union Operation

- Example: Find all courses taught in the Fall 2017 semester, or in the Spring 2018 semester, or in both
  - Result

$\Pi_{course_id}(\sigma_{semester=\text{“Fall”} \wedge year=2017}(teaches))$

course_id
CS-101
CS-347
PHY-101

$\Pi_{course_id}(\sigma_{semester=\text{“Spring”} \wedge year=2018}(teaches))$

course_id
CS-101
CS-315
CS-319
FIN-201
HIS-351
MU-199

Union Operation

- Example: Find all courses taught in the Fall 2017 semester, or in the Spring 2018 semester, or in both
  - Result

course_id
CS-101
CS-347
PHY-101

course_id
CS-101
CS-315
CS-319
FIN-201
HIS-351
MU-199

UNION

course_id
CS-101
CS-347
PHY-101
CS-315
CS-319
FIN-201
HIS-351
MU-199

Set-Intersection Operation

- The set-intersection operation finds tuples that are in both the input relations
  - Notation: $r \cap s$
  - Assumptions:
    - r, s have the same arity
    - Attributes of r and s are compatible

- Example: Find the set of all courses taught in both the 2017-Fall and 2018-Spring semesters
  - Query: $\Pi_{course_id}(\sigma_{semester=\text{“Fall”} \wedge year=2017}(teaches)) \cap$
    $\Pi_{course_id}(\sigma_{semester=\text{“Spring”} \wedge year=2018}(teaches))$

Set-Intersection Operation

- The set-intersection operation finds tuples that are in both the input relations
  - Notation: $r \cap s$
  - Assumptions:
    - r, s have the same arity
    - Attributes of r and s are compatible

- Example: Find the set of all courses taught in both the 2017-Fall and 2018-Spring semesters
  - Query: $\Pi_{course_id}(\sigma_{semester=\text{“Fall”} \wedge year=2017}(teaches)) \cap$
    $\Pi_{course_id}(\sigma_{semester=\text{“Spring”} \wedge year=2018}(teaches))$
  - Result

course_id
CS-101

Set-Difference Operation

- The set-difference operation finds tuples that are in one relation but are not in another
  - Notation: $r - s$
  - Assumptions:
    - r, s have the same arity
    - Attributes of r and s are compatible

- Example: Find all courses taught in the 2017-Fall semester, but not in the 2018-Spring semester
  - Query: $\Pi_{course_id}(\sigma_{semester=\text{“Fall”} \wedge year=2017}(teaches)) -$
    $\Pi_{course_id}(\sigma_{semester=\text{“Spring”} \wedge year=2018}(teaches))$

Set-Difference Operation

- The set-difference operation finds tuples that are in one relation but are not in another
  - Notation: $r - s$
  - Assumptions:
    - r, s have the same arity
    - Attributes of r and s are compatible

- Example: Find all courses taught in the 2017-Fall semester, but not in the 2018-Spring semester
  - Query: $\Pi_{course_id}(\sigma_{semester=\text{“Fall”} \wedge year=2017}(teaches)) -$
    $\Pi_{course_id}(\sigma_{semester=\text{“Spring”} \wedge year=2018}(teaches))$

course_id
CS-101
CS-347
PHY-101

course_id
CS-101
CS-315
CS-319
FIN-201
HIS-351
MU-199

Set-Difference Operation

- The set-difference operation finds tuples that are in one relation but are not in another
  - Notation: $r - s$
  - Assumptions:
    - r, s have the same arity
    - Attributes of r and s are compatible

- Example: Find all courses taught in the 2017-Fall semester, but not in the 2018-Spring semester
  - Query: $\Pi_{course_id}(\sigma_{semester=\text{“Fall”} \wedge year=2017}(teaches)) -$
    $\Pi_{course_id}(\sigma_{semester=\text{“Spring”} \wedge year=2018}(teaches))$
  - Result

course_id
CS-347
PHY-101

The Assignment Operation

- It is convenient at times to write a relational-algebra expression by assigning parts of it to temporary relation variables
  - Notation: $\leftarrow$
  - An assignment works like the assignments in a programming language

- Example: Find all instructor in the Physics and Music departments
  - Query: $Physics \leftarrow \sigma_{dept_name=\text{“Physics”}}(instructor)$

    $Music \leftarrow \sigma_{dept_name=\text{“Music”}}(instructor)$

    $Physics \cup Music$

- With the assignment operation, a query can be written as a sequential program
  - A sequential program consists of a series of assignments followed by an expression whose value is displayed as the result of the query

Rename Operation

- The results of relational-algebra expressions do not have a name that one can use to refer to them

- The rename operator, ρ, sets names to relational-algebra expressions
  - Notation: $\rho_{new_name}(E)$
    - Returns the result of expression *E* under the name, "*new_name*"

Equivalent Queries

- There is more than one way to write a query in relational algebra

- Example: Find information about courses taught by instructors in the Comp. Sci. department with salary greater than 50,000
  - Query 1:

    $$\sigma_{dept_name = \text{“Comp. Sci.”} \wedge salary > 50{,}000}(instructor)$$

  - Query 2:

    $$\sigma_{dept_name = \text{“Comp. Sci.”}}(\sigma_{salary > 50.000}(instructor))$$

  - The two queries are not identical; they are, however, equivalent -- they give the same result on any database

EOF

- Coming next:
  - MySQL
  - Structured Query Language